HEX BATTLES

AI Strategy — Complete Technical Reference

Source: artifacts/hex-strategy/app/game.tsx & utils/hexGrid.ts

1. Game Constants

1.1 Entities (EntityType)

Seven entity types exist. 'rebel' is a special neutral-owned entity that blocks income on its tile.

Entity Code	Display Name	Icon	Buy Cost	Base Upkeep	Strength	Is Unit?
simple_unit	Peasant	⚒	10	3	1	Yes
advanced_unit	Warrior	🗡	20	9	2	Yes
expert_unit	Swordsman	⚔	30	27	3	Yes
tower	Tower	🗼	15	1 per n	1	No
castle	Castle	🏰	30	5 per n	2	No
city	City	🏘	10	0	0	No
rebel	Rebel	✊	0	0	0	No (neutral)

> ℹ *Tower and castle base upkeep in ENTITY_META is per-building, but actual territory upkeep is progressive — see section 1.4.*

1.2 Terrain Income per Tile per Round

Terrain Type	Income / Round
grass	+2
forest	+2
desert	+1
mountain	0
lake	0

City Bonus (CITY_BONUS = 2): Added per tile when either `t.isCity === true` (native map city tile) OR entity on tile `=== 'city'` (player-built city).

// AGENT: It's not allowed to build a city on top of a native city.

1.3 Unit Upgrade Chain

From	To	Extra Cost	Extra Upkeep	Strength Gain
simple_unit	advanced_unit	+10	+6	1 → 2
advanced_unit	expert_unit	+10	+18	2 → 3

ℹ *Buildings cannot be upgraded. Upgrade path is one-directional and cannot skip a tier.*

1.4 Progressive Defense Upkeep Formulas

Tower: `total_upkeep = n × (n + 1) / 2` where n = number of towers in territory

Tower Count	1st	2nd	3rd	4th	5th
Cumulative Upkeep	1	3	6	10	15
Marginal Cost	1	2	3	4	5

Castle: `total_upkeep = 5 × n × (n + 1) / 2` where n = number of castles in territory

Castle Count	1st	2nd	3rd
Cumulative Upkeep	5	15	30
Marginal Cost	5	10	15

ℹ *These formulas apply to the TOTAL upkeep for all buildings of that type in the territory, not per building.*

1.5 Movement Costs

Terrain	Move Cost
grass, desert, lake	1
forest	2
mountain	Infinity (impassable)

Unit move range: 3 tiles per turn (standard). Partial moves track remaining range in `partialMoves`.

// AGENT: Forrest requires 2 movement cost.

2. Turn Structure

2.1 Overall Sequence

- Player ends their turn ('End Turn' button)

- runAiTurn() fires for all AI factions in order: ai1 → ai2 → ai3 → ai4 → ai5
- For each faction: iterate over all its contiguous territories (via BFS)
- For each territory: run the decision tree loop (max 100 iterations)
- State is mutated live — later territories see the results of earlier ones in the same AI turn

2.2 Round 1 — Special Behaviour

Round 1 permits only the free tower placement — all other AI actions are blocked.

- Each territory with ≥ 2 tiles may build exactly ONE free tower
- Placement algorithm: score every valid tile by counting owned-neighbour tiles in ZoC
- The tile with the highest score (most owned neighbours covered) wins
- Tie-break: first in array order
- Invalid placements: mountain, lake, any tile already holding an entity, or a graveyard tile
- Each territory tracks usage in freeTowerUsedTiles — prevents double use across turns
- After the free tower, the main decision loop is skipped (continue to next territory)

3. Economy

3.1 Income Calculation per Territory

- Iterate over every tile in the territory:
  - If entity on tile === 'rebel' → tile contributes zero income (rebel blocks it)
  - Otherwise: income += TERRAIN_INCOME[t.terrain]
  - If t.isCity === true: income += CITY_BONUS (2)
  - If entity on tile === 'city': income += CITY_BONUS (2)

3.2 Upkeep Calculation per Territory

- Count towers and castles: tower_count, castle_count
- Sum unit upkeep: Σ ENTITY_META[e].upkeep for each unit entity in territory
- Tower upkeep: tower_count × (tower_count + 1) / 2
- Castle upkeep: 5 × castle_count × (castle_count + 1) / 2
- Total upkeep = unit_upkeep + tower_upkeep + castle_upkeep

3.3 canAfford Rule

A purchase is permitted only when BOTH conditions hold simultaneously:

`currBal >= cost` — current balance covers the purchase price

`currIncome - (currUpkeep + extraUpkeep) >= 0` — income covers existing + new upkeep

ℹ *canAfford prevents the AI from buying things that would make the territory cash-flow negative. The balance acts as savings; income/upkeep is the recurring commitment.*

3.4 Income Modifier by Difficulty

Applied once per round at the end of the player's turn (inside handleEndTurn), before bankruptcy resolution:

Difficulty	incomeModifier per Round	Effect
super_easy	−territory.length	AI loses 1 gold per tile per round (economically crippled)
easy	0	No modifier
medium	0	No modifier
hard	0	No modifier
super_hard	+territory.length	AI gains 1 gold per tile per round (snowballs faster)

ℹ *incomeModifier applies ONLY at round-end recalculation. It does NOT affect the AI's internal canAfford check, so super_easy AI may sometimes overspend relative to its penalised income.*

3.5 Bankruptcy Cascade

At round end:

```
newBalance = balance + income + incomeModifier - upkeep
```

If newBalance < 0:

- Territory balance is zeroed to 0
- PASS 1 — Kill all units (isUnit === true): move to graveyard, accumulate unitUpkeepSaved
- PASS 2 — If delta + unitUpkeepSaved is still < 0: demolish all buildings except 'rebel' and 'city' → ruins
- Cities (city) and rebels (rebel) are always exempt from demolition

4. Combat and Movement

4.1 Zone of Control (ZoC)

An entity at tile X with strength S exerts ZoC strength S on X and all 6 adjacent tiles.

`getMaxEnemyZoC(targetKey, aiOwner, ...)` returns the HIGHEST strength value exerted by any non-AI entity on or adjacent to the target.

Attack rule: A unit with strength S can only move to a tile where enemy ZoC < S.

Attacker Strength	Can beat ZoC ≤	Cannot attack ZoC ≥	Note
1 (Peasant)	0 (empty)	1	Can only take undefended tiles
2 (Warrior)	1	2	Beats towers and peasants
3 (Swordsman)	2	3	Cannot capture a castle-defended tile

> ℹ *A castle provides ZoC strength 2. A swordsman (strength 3) exerts ZoC 3 — so a castle-defended tile can only ever be taken by another castle or swordsman ZoC. Strength-3 units can never directly capture a castle-protected tile via movement.*
> ***// AGENT: All of this is wrong. A Castle is STR 2 and can therefore be bested by a STR 3 Swordsman. Fix this bug.***

4.2 Valid Movement Targets (getValidMoves)

- BFS from starting tile, budget 3 movement points (standard move range)
- Mountain tiles: impassable (Infinity cost), cannot be entered or passed through
- Friendly tiles: traversable and can be the final destination
- Friendly city tiles: traversable and can be the final destination
- Enemy/neutral tiles: valid landing target only if attacker strength > ZoC on that tile
- Spent units (in spentUnits set): cannot move again this turn

// AGENT: Friendly tiles and Friendly city tiles can be final destinations.

4.3 Lake Movement (Lake Transfer)

Pre-check (performed BEFORE any state mutation):

- If destination is lake AND srcBalance < unitUpkeep × 2 → return false, no state changes

On successful lake entry:

- lakeAmount = min(unitUpkeep × 2, 15) is deducted from the source territory's balance
  //AGENT: Always just bring 2x upkeep of unit. No max of 15
- lakeAmount is stored in lakeUnitFunds[lakeKey]
- Unit is placed on the lake tile and marked as spent

On shore landing (lake → land):

- lakeUnitFunds for the lake tile are returned to the destination territory's balance
- lakeUnitFunds[lakeKey] is cleared

> ℹ *The AI only moves onto a lake tile when dtLakeHasLandTarget() confirms a reachable land tile exists on the far side. This prevents stranding.*
>
> *// AGENT: Make sure that the embarking unit is able to conquer (S > ZoC) the reachable land before moving the unit on to the lake. If not, stay on your own territory.*

4.4 City Capture

When a unit moves to a tile occupied by a enemy/neutral city entity:

- The city entity is PRESERVED at the tile (ownership transferred to new owner)
- The capturing unit is consumed (deleted from entities — it is 'sacrificed')
- The tile changes owner — city income (+2/round) accrues to the new owner from next round

ℹ *This is intentional game design: one unit is the cost to capture a city. Both player and AI follow the same rule.*

// AGENT: “The capturing unit is consumed (deleted from entities — it is 'sacrificed')” this is wrong. Delete that.

4.5 Unit Merging

When a unit moves to a friendly tile that already has a unit:

Moving Unit	Tile Unit	Result	New Entity Code
Strength 1	Strength 1	Strength 2 (merge)	advanced_unit
Strength 2	Strength 1	Strength 3 (merge)	expert_unit
Strength 1	Strength 2	Strength 3 (merge)	expert_unit

ℹ *Merging to strength > 3 is impossible — the tile cannot be entered. A strength-3 unit tile is also an impassable friendly destination.*

5. AI Decision Logic

5.1 State Determination: Attacking vs. Defending

DEFENDING: There exists at least one enemy unit (isUnit=true) within hexDistance ≤ 3 of ANY border tile of the current territory, AND that unit's strength > currMaxStr (the AI's own maximum entity strength in this territory).

ATTACKING: Default state when no stronger-than-self enemy threat is within 3 tiles of any border tile.

ℹ *strongerEnemy = the single highest-strength qualifying threat. Neutral tiles do not count as threats. 'Border tile' = any AI tile adjacent to a non-AI, non-neutral tile.*

// AGENT: The stronger unit we’re looking for has to be on an adjacent territory, up to 3 tiles away, not just any territory.

5.2 Difficulty Skip-Chance

Checked at the START of each decision-loop iteration before any priority is evaluated:

Difficulty	Skip Chance	Mechanism
super_easy	0%	No skip — handicapped via income modifier (section 3.4) instead

Difficulty	Skip Chance	Mechanism
easy	40%	Math.random() < 0.4 → skip entire iteration for this territory
medium	20%	Math.random() < 0.2 → skip entire iteration for this territory
hard	0%	Never skips — executes all possible actions every iteration
super_hard	0%	Never skips — plus receives income bonus each round

ℹ *A skip discards the ENTIRE iteration — no movement, purchase, or build occurs. The next iteration restarts from priority DEF-1. Multiple consecutive skips are possible.*

// AGENT: Super Easy has to have the same skip change as Easy

5.3 Decision Priorities (Strict Order)

Priorities are evaluated in the sequence below. The FIRST actionable priority executes one action and restarts the loop. Only DEF-1 and DEF-2 are gated on DEFENDING state.

// AGENT: Make sure this is considered for every action that involves units "Evaluate for each valid action: moving an existing unit, merging 2 units and moving, or buying a new unit directly into an attack". Make sure to check it in that order in every of the following priorities.

DEF-1 (DEFENDING only): Split the Threatening Enemy

- Condition: currAiState === 'defending' && strongerEnemy !== null
- For each available own unit: compute getValidMoves(...)
- Filter targets: tile belongs to strongerEnemy.owner, attacker strength > ZoC, lake targets need a land exit
- Score each candidate:
  - score = dtSplitScore(): number of separate enemy components after capture (only > 0 counts)
  - neg = dtCaptureNegatesIncome(): would capture make the enemy's balance+delta < 0?
  - oneHex = dtCaptureCreatesOneHex(): does capture leave any enemy component with exactly 1 tile?
- Sort: oneHex=true first, then neg=true, then highest score
- Only candidates with score > 0 OR neg OR oneHex qualify
- Action: dtExecMove(best candidate)

- FALLBACK (if no candidate qualifies): find the enemy tile whose capture minimises the total size of enemy components that still contain units — attack that tile

DEF-2 (DEFENDING only): Reinforce Against the Threat

- Condition: currAiState === 'defending' && strongerEnemy !== null
- eStr = strongerEnemy.strength

Step 1 — Upgrade an existing building to match threat strength:

- Find buildings (non-units) in territory with a valid UNIT_UPGRADE path to strength >= eStr
- upgCost = upgraded.cost − current.cost, dUpk = upgraded.upkeep − current.upkeep
- Requires: canAfford(upgCost, dUpk)
- Action: dtExecUpgrade(tile, upgradedType, upgCost)

// AGENT: Make sure that tower are only upgraded if there is a S > ZoC unit in an adjacent territory and up to 3 tiles away.

Step 2 — Upgrade an existing unit within 5 tiles of the threat:

- Filter availUnits to hexDistance(unit, strongerEnemy.key) ≤ 5
- Unit must have upgrade path to strength >= eStr
- Requires: canAfford(upgCost, dUpk)
- Action: dtExecUpgrade(unitKey, upgradedType, upgCost)

Step 3 — Build a new building with strength >= eStr within 5 tiles of threat:

- Try building types in order: [castle, tower]
- Skip if ENTITY_META[type].strength < eStr
- Castle extra condition: territory must already have at least one advanced_unit or expert_unit
- Valid placements: not mountain/lake, not occupied, hexDistance to threat ≤ 5
- Sort placements: closest to threat first
- Action: dtExecBuild(type, closestPlacement, cost)

Step 4 — Build a building with strength >= eStr−1 directly on a border tile:

- Try building types: [tower, castle]
- Condition: ENTITY_META[type].strength >= eStr − 1
- Valid placements: border tiles (adjacent to non-AI, non-neutral), not mountain/lake, not occupied
- Action: dtExecBuild(type, borderPlacement, cost)

// AGENT: Step 4, prefer to place the weaker tower 1 tile away from the border tile.

Priority A (All States): Tactical Split

- Find all valid move targets on ENEMY (non-neutral) tiles for all available own units
- Filter: attacker strength > ZoC, lake targets need land exit
- Compute score, neg, oneHex for each candidate (same formulas as DEF-1)
- Only candidates with score > 0 OR neg OR oneHex qualify
- Sort: oneHex > neg > highest score
- Action: dtExecMove(best candidate)

Priority B (All States): Bridge Own Territories

Find tiles that form a 'bridge' between the current territory and another owned territory:

1-tile bridge: An unowned adjacent tile that is ALSO adjacent to a different AI territory.

2-tile bridge: An adjacent tile → its own adjacent tile, which is adjacent to another AI territory.

- Mountain and lake tiles are invalid bridge tiles

- Sort bridges: prefer connecting to the LARGEST other AI territory
- Attempt 1: move an existing unit to the bridge tile (requires strength > ZoC)
- Attempt 2: buy a new unit adjacent to the bridge tile — units tried in order: simple_unit, advanced_unit, expert_unit
    - Condition: canAfford(cost, upk) AND strength > ZoC AND a current-territory tile is adjacent to bridge tile
- Action: dtExecMove or dtExecBuy

Priority C (All States): Defend Unprotected Border Tiles

- Unprotected border tiles: border tiles with NO entity, or only a rebel entity
- Condition: at least one adjacent enemy territory has ≥ 2 tiles (not just a lone tile)
- Preferred placements: inner tiles exactly 1 hex from an unprotected border tile (avoids placing on the border itself)
- Fallback placements: the border tiles themselves if no inner tile is available
- Try building types in order: [tower, castle]
- Castle extra condition: territory must have at least one advanced_unit or expert_unit
- Valid placements: not mountain/lake, not occupied
- Requires: canAfford(cost, upk)
- Action: dtExecBuild(type, placement, cost)

// AGENT: "Unprotected border tiles: border tiles with NO entity, or only a rebel entity" use ZoC to determine if a tile is unprotected. A tile next to a unit or a tower/castle is considered protected.
// AGENT: Place tower/castle 3 tiles away from another tower/castle, to avoid overlapping. If this is not possible place it 2 tiles away. Never place it right next to another Tower/castle.

Priority D (All States): Build a City

- Conditions (ALL must hold):
    - canAfford(10, 0) === true
    - currTerr.length >= 6 (territory has at least 6 tiles)
    - !alreadyHasCity (no existing isCity tile AND no city entity anywhere in territory)

Placement algorithm:

- Compute bldgZoC: the set of all tiles within ZoC of any own tower or castle
- Candidate tiles: inside bldgZoC, not mountain/lake, not an existing isCity tile, not occupied
- If no candidate qualifies, action is skipped
- Sort candidates: furthest from the LARGEST enemy territory (maximises safety margin)
- Action: dtExecBuild('city', bestCandidate, 10)

Priority E (All States): Border Expansion

E1 — Attack enemy tile with unit or building:

- Gather all valid moves landing on enemy tiles that have an entity (excluding rebel)
- Filter: attacker strength > ZoC, lake → land exit required
- Sort: highest enemy entity strength first (take out the biggest threat)

- Action (move): dtExecMove(best)

E1 buy — Purchase a unit to attack enemy entity:

- Try types in order: [expert_unit, advanced_unit, simple_unit] (strongest first)
- For each type: find an enemy tile adjacent to an own tile that has an enemy entity (not rebel)
- Condition: canAfford(cost, upk) AND strength > ZoC AND lake needs land exit
- Action: dtExecBuy(type, enemyTile, cost, isOutside=true)

E2 — Capture empty enemy tile:

- Gather all valid moves landing on enemy tiles with NO entity (or only rebel)
- Filter: attacker strength > ZoC, lake → land exit required
- Sort: prefer tiles belonging to the LARGEST enemy territory
- Action (move): dtExecMove(best)

E2 buy — Purchase a unit adjacent to an empty enemy tile:

- Try types: [simple_unit, advanced_unit, expert_unit]
- Select the enemy tile with highest sz (largest enemy territory) where strength > ZoC
- Action: dtExecBuy(type, enemyTile, cost, isOutside=true)

E3 — Capture neutral tile:

- Gather all valid moves landing on neutral tiles
- Filter: attacker strength > ZoC, lake → land exit required
- **Priority order:** `t.isCity === true` **(3) >** grass/forest **(2) >** desert **(1)**
- Action (move): dtExecMove(highest priority)

E3 buy — Purchase a unit to capture neutral tile:

- Try types: [simple_unit, advanced_unit, expert_unit]
- Select highest-priority neutral tile adjacent to own territory where strength > ZoC
- Action: dtExecBuy(type, neutralTile, cost, isOutside=true)

// AGENT: Make sure priority E is carried out always before moving on the Priority F, even if it just capturing a lonely tile.

Priority F (All States): Rebel Suppression

- Find all tiles in current territory where entity === 'rebel'

F1 — Move an existing unit onto the rebel tile:

- Sort own units weakest-to-strongest (use minimum force)
- Try each unit in order — pick the first that can reach the rebel tile
- Action: dtExecMove(weakestReachingUnit, rebelTile)

F2 — Buy a simple_unit directly on the rebel tile:

- Condition: currBal >= 10 AND currIncome + rebelTileIncome − (currUpkeep + 3) >= 0

- rebelTileIncome = TERRAIN_INCOME[terrain] + (isCity ? 2 : 0)
- Action: dtExecBuy('simple_unit', rebelTile, 10, isOutside=false)

Priority G (All States): March Units Toward Enemy

- Collect all enemy tiles (not mountain, not lake)
- For each unspent unit NOT already adjacent to an enemy tile:
    - Find the nearest enemy tile by hexDistance
    - Attempt BFS step: dtBfsStep(unit, nearestEnemyTile, validMoves) → one-step toward target
    - Fallback (BFS returns null): greedy pick — the valid move that minimises distance to ANY enemy tile
- Action: dtExecMove(unit, bestStep)

ℹ *Units already adjacent to an enemy tile are skipped — they are handled by priority E.*

Priority H (All States): Demolish Distant Defense Buildings

- For each tower or castle owned by this AI in the current territory:
- Find every 'active border tile': a border tile adjacent to an enemy territory with ≥ 2 tiles
- Compute minDist = minimum hexDistance from the building to any active border tile
- Condition: minDist > 5 (building is far from any real threat)
- Action: dtExecRemove(buildingKey) — entity deleted, upkeep freed, no ruins/graveyard entry

ℹ *Border tiles facing only 1-tile territories are NOT counted as active borders. Isolated single-tile fragments do not justify keeping distant buildings.*

// AGENT: Change this to only remove towers if there are no enemy tiles within a 6 tile radius.

6. Internal AI Helper Functions

Function	Purpose
dtSplitScore(captureKey, owner)	Simulates capture and counts resulting separate enemy territory components. Returns 0 if no split occurs.
dtCaptureNegatesIncome(key, owner)	Returns true if capture would leave the enemy territory with balance + (income − upkeep) < 0 (bankruptcy next round).
dtCaptureCreatesOneHex(key, owner)	Returns true if capture would leave any enemy component with exactly 1 tile (auto-bankrupt at round end).
dtLakeHasLandTarget(lakeKey)	Returns true if a unit on this lake tile can reach at least one non-lake, non-mountain tile within move range.
dtBfsStep(from, to, validMoves)	BFS pathfinding from 'from' toward 'to'. Returns the FIRST tile on the shortest path, or null if unreachable.

6.1 dtExecMove — Mutation Sequence

- PRE-CHECK lake: if destination is lake AND srcBal < unitUpkeep × 2 → return false (no mutations)
- Determine terrain of destination (lake or land)
- Update workingTileMap: set destination tile owner to aiOwner
- Handle city capture: if destination entity === 'city' → delete the moving unit, keep city entity
- Handle merge: if destination has a friendly unit → compute mergedUnitType(strength1, strength2), place merged unit
- Default: delete entity at source, place unit at destination
- Lake entry: lakeAmount = min(upkeep × 2, 15); deduct from source balance; store in lakeUnitFunds
- Shore landing: add lakeUnitFunds[lakeKey] to destination territory balance; clear lake funds
- Mark source key as spent in workingSpentUnits
- Call recalculateTerritoriesForCapture() — updates balances for newly merged/split territories
- Call applySingleHexPenalty() — kills units in newly isolated 1-tile territories

6.2 dtExecBuy

- targetKey must not already have an entity (collision prevention)
- targetKey terrain must not be mountain or lake
- isOutsideTerritory=true: AI does not own the tile but it is adjacent to own territory
- If isOutside: update tileMap ownership, call recalculateTerritories
- Deduct cost from workingBalances for the relevant territory
- Place entity on targetKey in workingEntities

6.3 dtExecBuild

- Target tile must be in AI's territory
- Target terrain must not be mountain or lake
- Target must not already hold an entity
- City extra checks: territory has ≥ 6 tiles, no existing city in territory
- Deduct cost from workingBalances
- Place building entity on tile

6.4 dtExecUpgrade

- Target tile must be in AI's territory and hold an upgradeable entity
- Deduct upgCost (difference between new and old cost) from balance
- Replace entity with upgraded type

6.5 dtExecRemove (Priority H only)

- Target must be a tower or castle in AI's territory

- Entity is deleted — no ruins entry, no graveyard entry, upkeep is freed immediately

7. Known Limitations and Notes for Revision

#	Issue	Location	Impact
1	canAfford ignores incomeModifier — super_easy AI may overspend relative to its penalised actual income	game.tsx: canAfford closure	Minor: super_easy may remain solvent longer than intended
2	Skip-chance fires per iteration, not per action — consecutive skips possible	game.tsx: line ~1657	By design — but extreme consecutive skips on easy are possible
3	DEF-1 fallback (minimal-area attack) is greedy and sub-optimal	game.tsx: lines ~1692–1720	AI may attack the wrong tile when no clean split exists
4	Priority B only buys units for bridges, never builds a tower/castle to hold one	game.tsx: lines ~1900–1914	Bridges can be captured back next turn if undefended
5	Priority G: 'already adjacent' check uses current position, not post-move	game.tsx: lines ~1186–1193	Units may stop one tile short of acting on priority E
6	Priority H ignores 1-tile enemy territories when computing active border distance	game.tsx: line ~2238	Correct by design — 1-tile fragments bankrupt automatically
7	E1 buy always tries expert_unit first regardless of ZoC requirement	game.tsx: lines ~2023–2046	May waste gold on over-strength unit when a cheaper one would do

— End of AI Strategy Reference —